Route GEMM

FP32-weight router GEMM on BF16 Tensor Cores

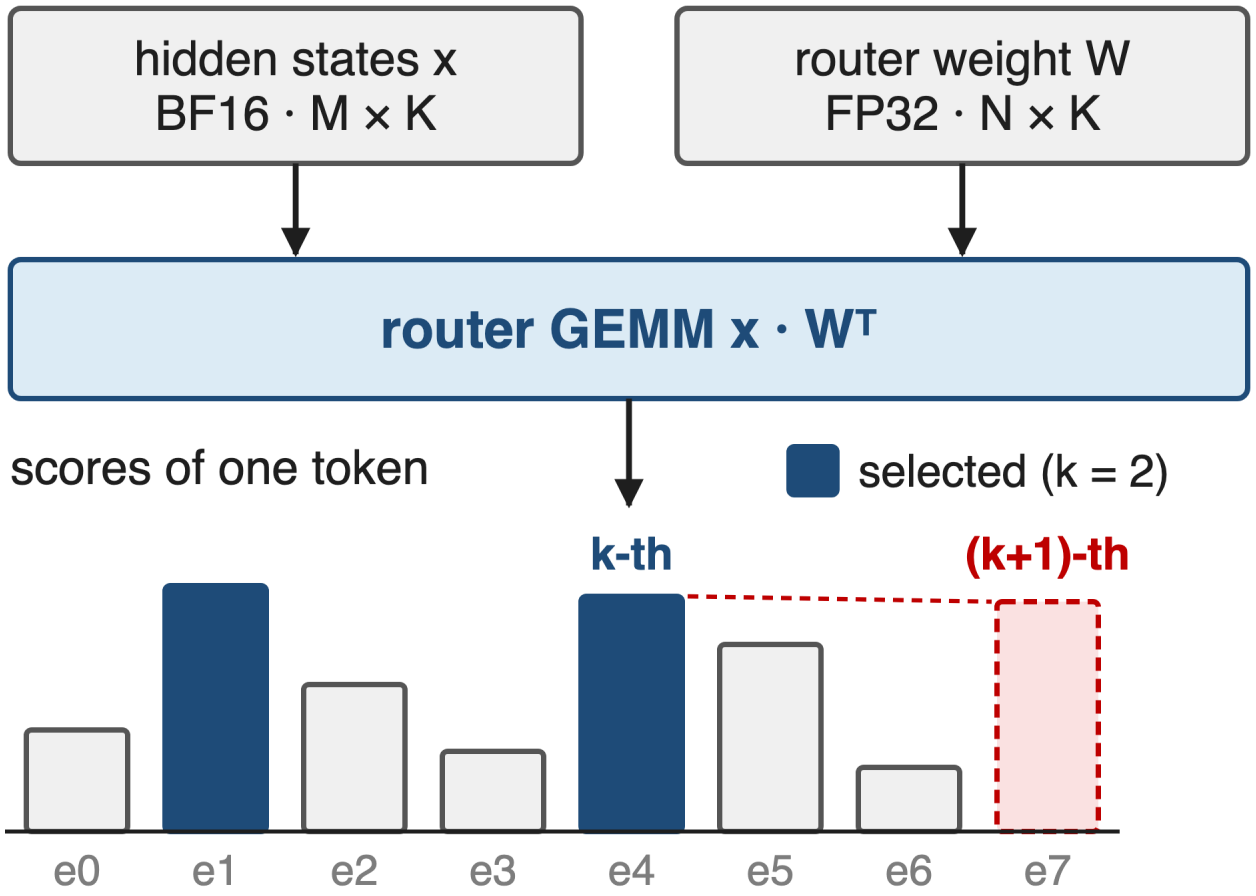
Tencent Hunyuan AI Infra · HPC-Ops

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

GEMM & MoE Kernels · github.com/Tencent/hpc-ops

Router scores pick each token's experts

- Router GEMM: BF16 tokens × **FP32** weights
- LongCat-Flash router: K = **6144**, N = **768**
- Same shape of problem: state-compress GEMMs in sparse and linear attention



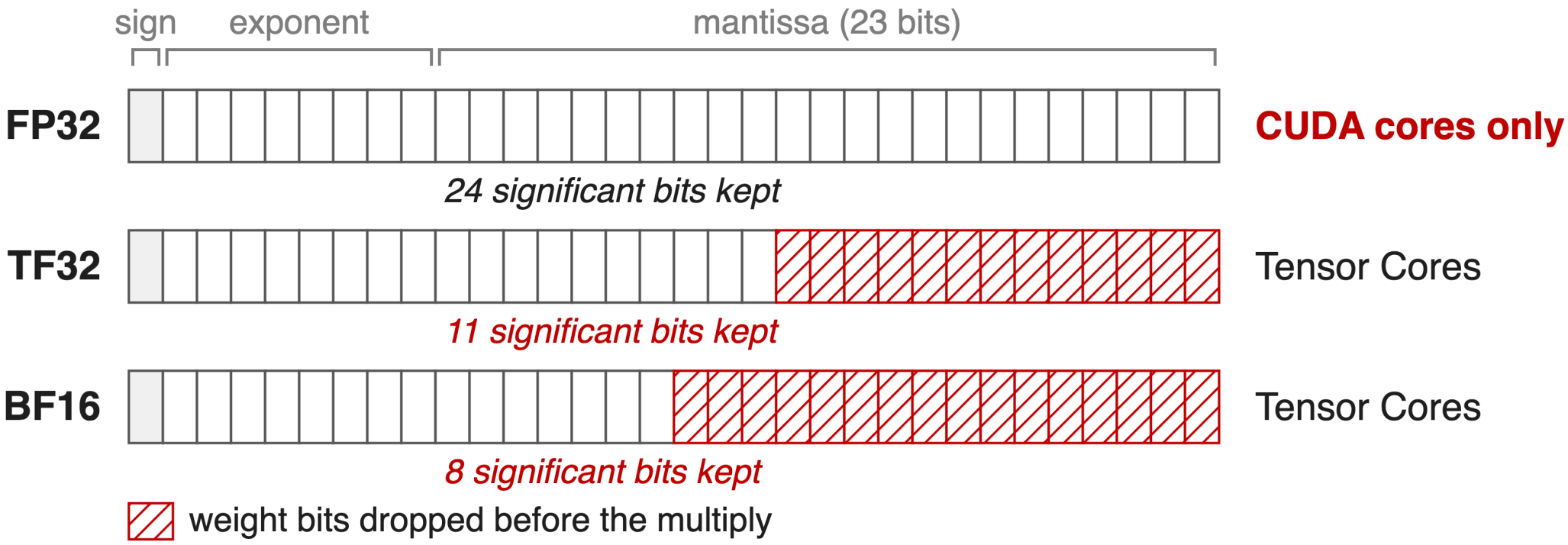
Exact FP32 weights cost 1.96× TF32's time

1.96×

FP32 cuBLAS takes 403.7 µs, TF32 cuBLAS 205.8 µs, for the same router GEMM (M = K = 4096, N = 192, H20)

Derived from LMSYS "HPC-Ops × SGLang" (Aug 2026), Table 2; both baselines include the x.float() cast

Tensor-Core formats drop the weight's low bits



Exact but slow, or fast but rounded: cuBLAS offers nothing in between

Insight: one FP32 weight is two BF16 weights

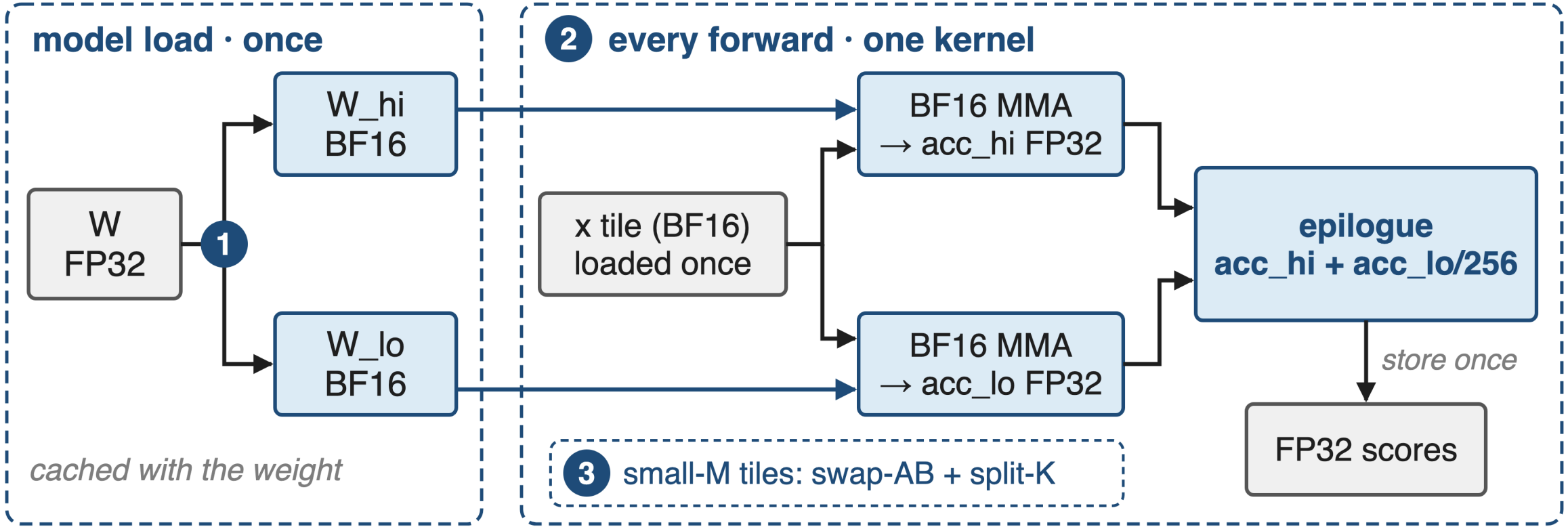
- $w \approx w_hi + w_lo / 256$, both stored in BF16
- A GEMM is linear: $x \cdot w = x \cdot w_hi + (x \cdot w_lo) / 256$

Insight: one FP32 weight is two BF16 weights

- $w \approx w_{hi} + w_{lo} / 256$, both stored in BF16
- A GEMM is linear: $x \cdot w = x \cdot w_{hi} + (x \cdot w_{lo}) / 256$

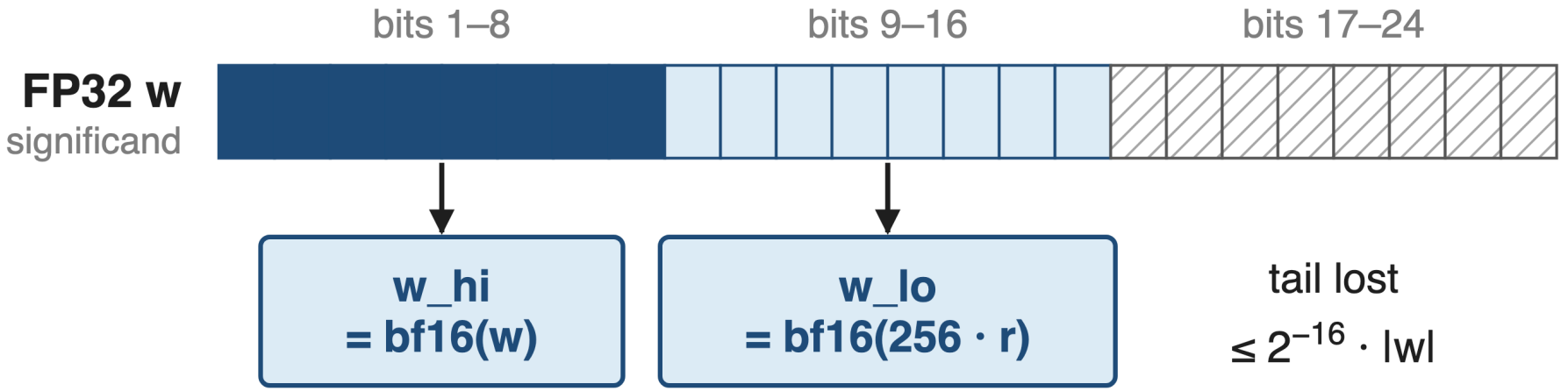
Two BF16 Tensor-Core GEMMs recover 16 of the weight's 24 bits

Split once, multiply twice, store once



API: hpc.gemm_bf16xfp32(x, w_high, w_low, scale) · kernel: src/gemm/sm90/gemm.cu

❶ The split keeps bits 1–16 of the weight



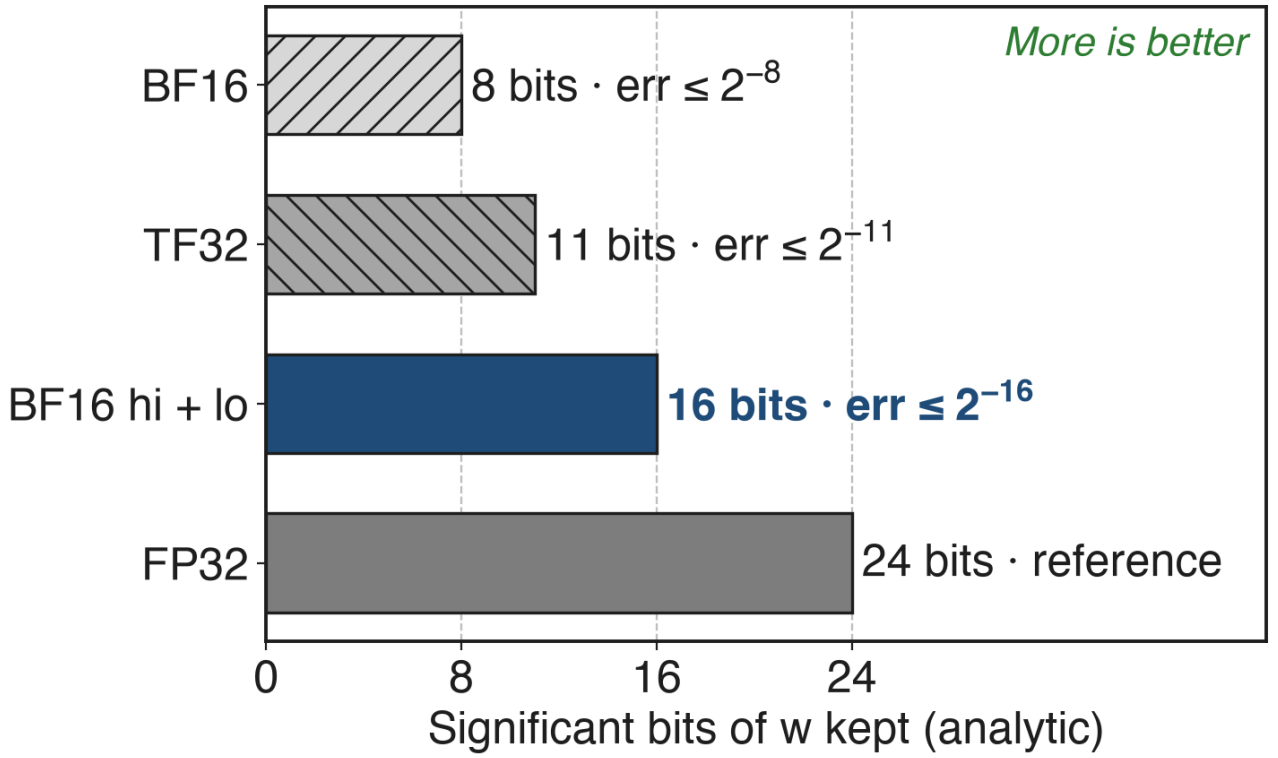
example

```
w = 0.1234567        w_hi = 0.1235352          r = -7.85e-5
w_lo = -0.02014      w_hi + w_lo/256 = 0.1234565   error = 2.2e-7
```

256 = 2^8 lifts bits 9–16 into BF16's 8-bit window

Only the dropped tail bits cost accuracy

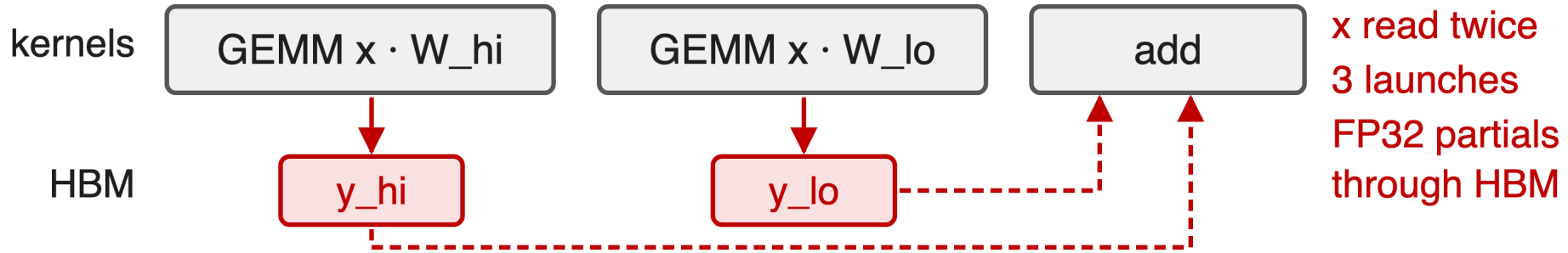
- x is BF16: no input rounding
- BF16 × BF16 products exact; FP32 accumulators
- ×1/256 is exact (power of two)



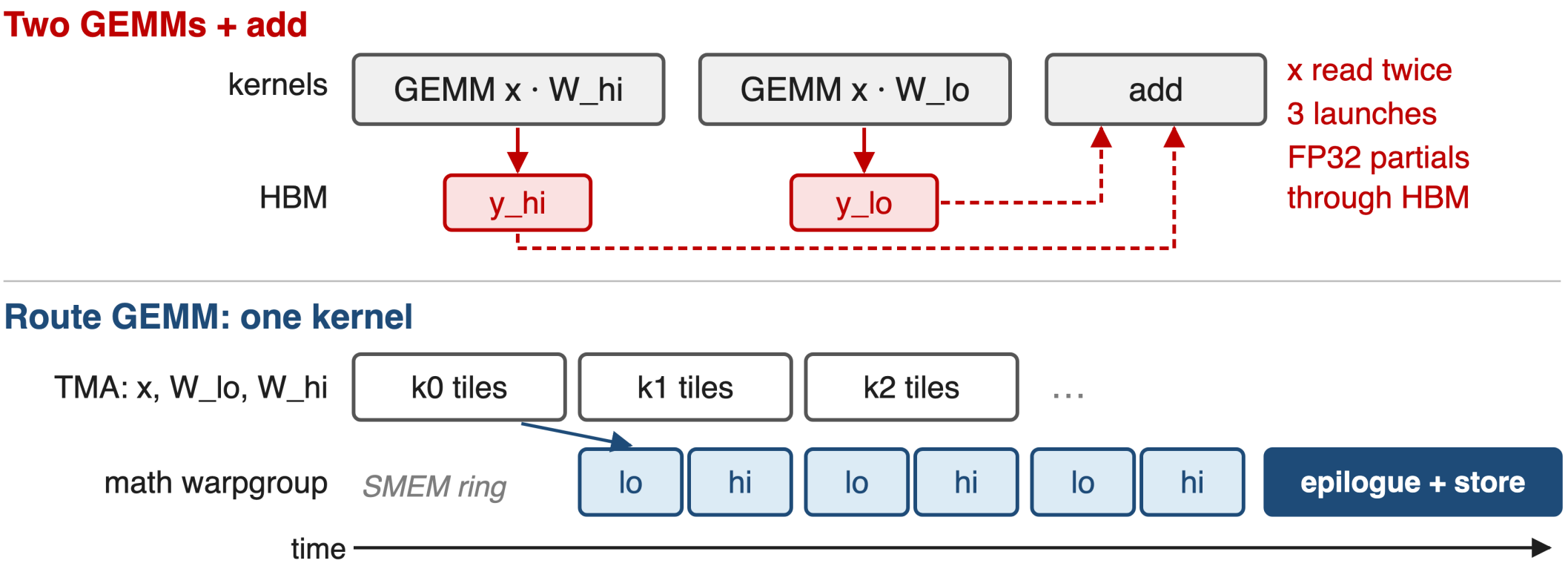
Weight error ≤ 2^{-16}·|w|: 32× finer than TF32

❷ One kernel feeds both GEMMs from one tile

Two GEMMs + add

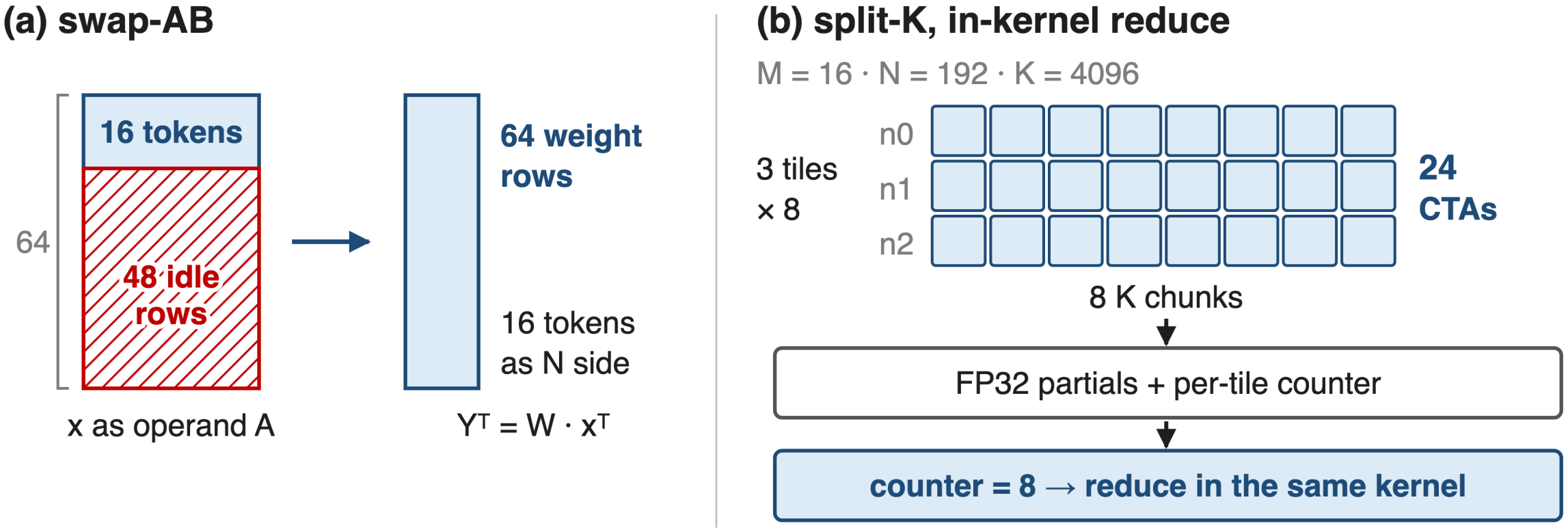


❷ One kernel feeds both GEMMs from one tile



Two FP32 accumulators stay in registers until one final store

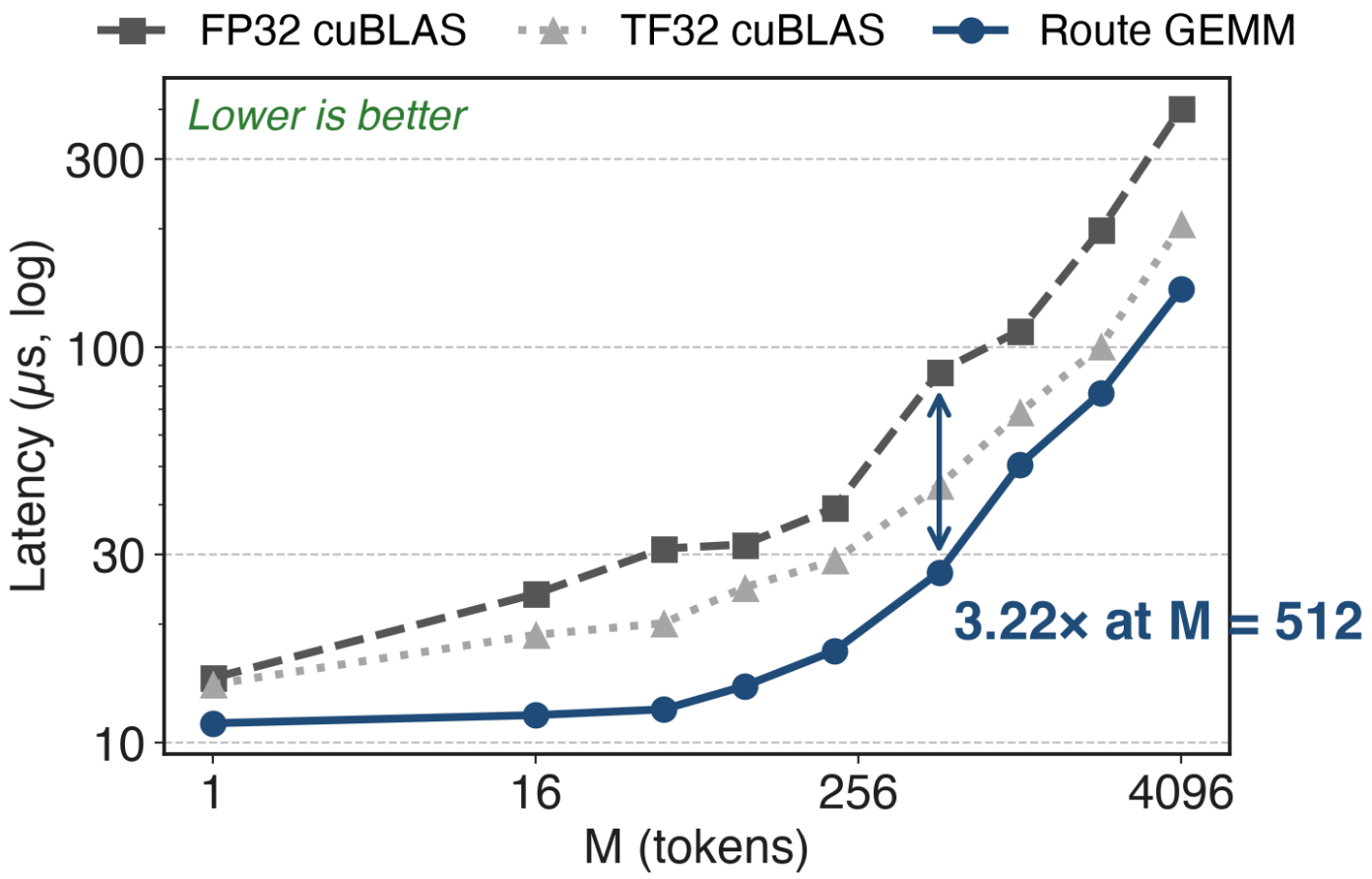
❸ Swap-AB and split-K fill the GPU at small M



Picker (entry.cc): split-K 8 / 4 / 2 / 1 as normalized M grows; 64-token tiles above it

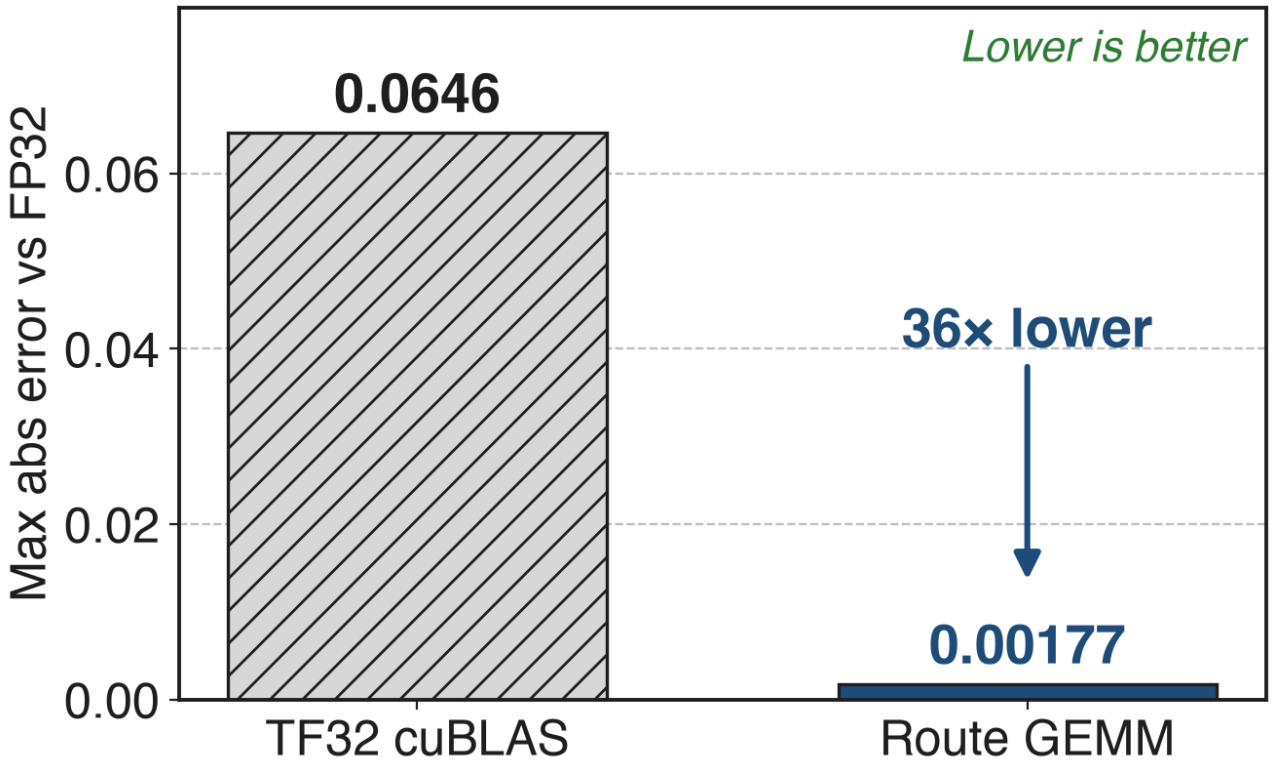
Beats FP32 cuBLAS by 1.30–3.22× on H20

- H20 · router shape K = 4096, N = 192
- **1.25–1.78×** faster than TF32 cuBLAS too
- Baselines include the `x.float()` cast



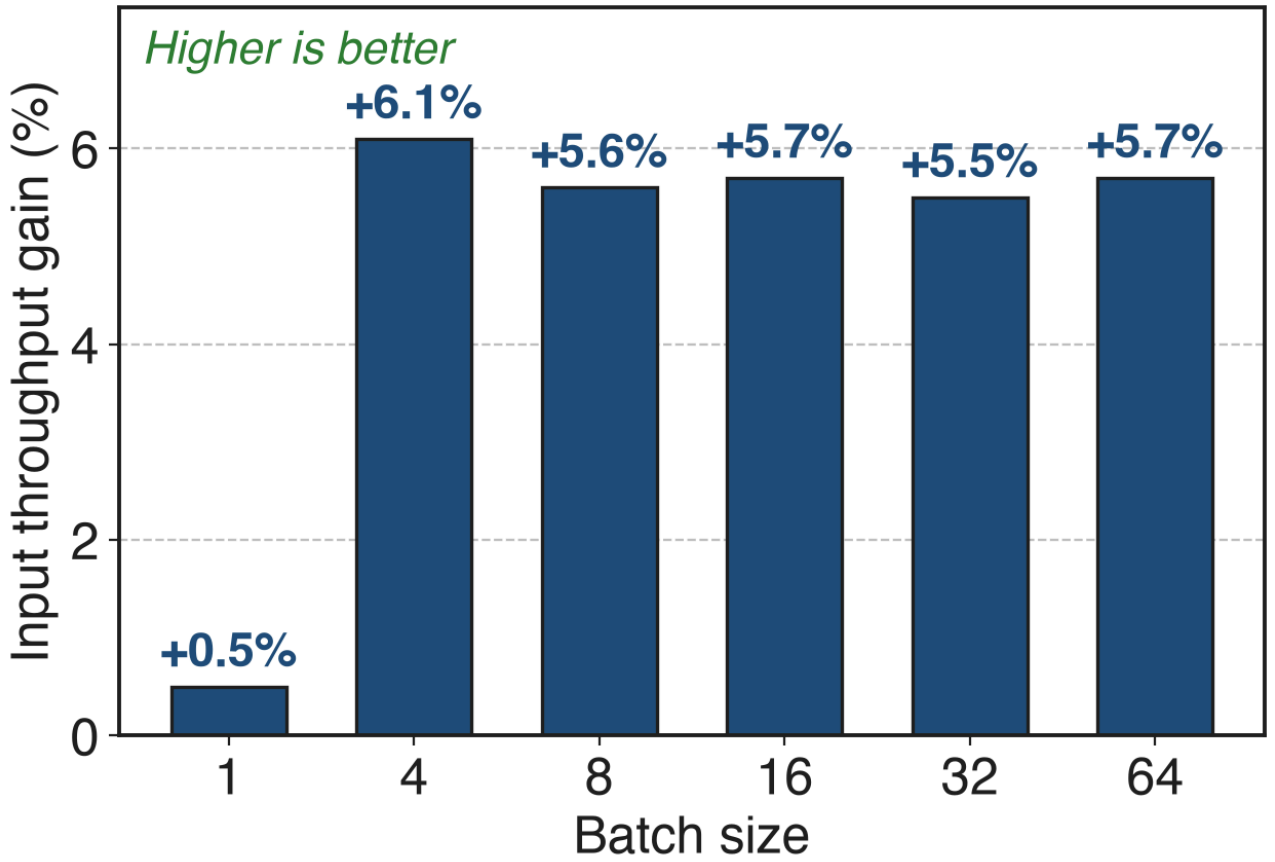
Error stays 36× below TF32 at higher speed

- Max abs error vs FP32 cuBLAS, same sweep
- Analytic bound predicts **32×** (2^{-16} vs 2^{-11})
- SGLang: greedy outputs unchanged



LongCat prefill gains 5.5–6.1% input throughput

- LongCat-Flash-Lite-FP8, 8×H20, router GEMM only
- Router shapes: **1.06–2.83×** over SGLang default
- GSM8K **0.798** → **0.802** (H200)



The cost: crossovers, frozen weights, Hopper only

- **Below a crossover M**, cuBLAS FP32 is faster
- SGLang min-M guard **64** / **128**: Lite decode keeps cuBLAS
- ~16-bit weights: close to FP32, not bit-exact

- **Online weight updates rejected** while the split cache is active
- SM90 only; N divisible by 64; forward only

Summary

- Exact FP32 router weights cost **1.96×** TF32's time
- **Insight**: w ≈ w_hi + w_lo/256 turns one FP32 GEMM into two BF16 GEMMs
- One kernel: shared x tile, two FP32 accumulators, one store
- **1.30–3.22×** vs FP32 cuBLAS, error **36×** below TF32

https://github.com/Tencent/hpc-ops